# API própria para a tomada NovaDigital T20A

Guia de implantação piloto em um ambiente

Objetivo: ler uma tomada na rede local, verificar os campos realmente disponíveis, registrar amostras e expor uma API HTTP de leitura. O piloto não envia comandos para ligar ou desligar.

## Arquitetura do piloto

| Origem | Ligação | Destino |
|---|---|---|
| Tomada T20A | Protocolo Tuya na LAN | Home Assistant + tuya-local |
| Home Assistant | REST API autenticada | API Python do piloto |
| API Python | Amostras a cada 60 s | SQLite + consultas HTTP |

## Resultado esperado

- Uma URL para a leitura atual e outra para o histórico de amostras.
- Dados com valor, unidade e horário informados pelo Home Assistant.
- Inventário dos campos que a unidade piloto realmente fornece.

Base técnica: o perfil público de tuya-local cita NovaDigital T20A, produto Tuya `keykmm3rjyqy5r8p`, e prevê estado, energia, corrente, potência e tensão. Parte desses campos é opcional; a unidade precisa ser testada.

## 1. Preparar o ambiente

- Escolha uma tomada piloto e um computador ou VM que fique ligado na mesma rede local. Reserve endereço IP para ambos no DHCP.
- Garanta acesso do computador à tomada na LAN. Redes de convidados e VLANs isoladas podem impedir a descoberta ou a conexão.
- Tenha acesso à conta do aplicativo e a um computador Linux com Python 3.10+ para a API. Não reinicie nem recadastre uma tomada que alimente equipamento crítico.

## 2. Instalar o Home Assistant

Instale Home Assistant OS em mini PC ou VM conforme o guia oficial. Para VM, use pelo menos 2 vCPU e 2 GB de RAM; conecte a VM à rede que alcança a tomada. Abra a interface pelo IP do servidor, geralmente na porta 8123, e conclua o cadastro inicial.

Guia oficial: home-assistant.io/installation/linux

## 3. Instalar HACS e tuya-local

- No Home Assistant OS, adicione o repositório de apps `https://github.com/hacs/addons`. Em Configurações > Apps, instale "Get HACS", execute-o, siga o log e reinicie o Home Assistant.
- Finalize a configuração do HACS em Configurações > Dispositivos e serviços. No HACS, procure "Tuya Local" do repositório `make-all/tuya-local` e instale a integração.
- Reinicie o Home Assistant. Em Configurações > Dispositivos e serviços > Adicionar integração, selecione "Tuya Local".

Instruções atualizadas: HACS e tuya-local.

## 4. Cadastrar a tomada

- Na integração, escolha o fluxo assistido pelo aplicativo Tuya/Smart Life, entre na conta que contém a tomada e selecione o dispositivo. Esse fluxo obtém o ID e a chave local sem criar projeto IoT Developer.
- Se as tomadas estiverem apenas no aplicativo NovaDigital e não aparecerem nesse fluxo, pare nesta etapa: verifique a opção de cadastro manual com ID e chave local. Migrar uma unidade para Smart Life exige recadastramento e pode interromper suas automações.
- Confirme que a tomada aparece como NovaDigital T20A ou compare o identificador de produto com `keykmm3rjyqy5r8p`. Se não coincidir, registre o identificador real e continue apenas com os campos detectados.

## 5. Verificar os dados reais

Abra a página do dispositivo no Home Assistant e anote os IDs das entidades exibidas. Em Ferramentas do desenvolvedor > Estados, observe o valor e a unidade enquanto uma carga conhecida é ligada à tomada. Habilite entidades ocultas ou desativadas, se existirem.

| Leitura | DP esperado | O que confirmar |
|---|---|---|
| Ligada | 1 | on/off muda ao acionar a tomada. |
| Energia | 17 | Aparece e aumenta com uma carga; anote unidade. |
| Corrente | 18 | Valor reage à carga; anote unidade. |
| Potência | 19 | Valor reage à carga; anote unidade. |
| Tensão | 20 | Valor plausível; anote unidade. |

Critério de avanço: obtenha pelo menos estado e potência; para consumo em kWh, confirme a energia do DP 17 ou planeje cálculo a partir das amostras de potência. Uma leitura estimada a partir de W não deve ser tratada como medição para faturamento.

## 6. Criar token do Home Assistant

- No perfil do usuário do Home Assistant, crie um "Long-Lived Access Token" para a integração. Guarde-o em gerenciador de segredos; ele dá acesso à API do Home Assistant.
- Teste um ID de entidade com a API REST. Substitua os valores entre sinais <>:

```
curl -H "Authorization: Bearer <HA_TOKEN>" \
+  "http://<IP_HA>:8123/api/states/<ENTIDADE>"
```

Resposta esperada: JSON com `state`, `attributes.unit_of_measurement` e `last_updated`. Referência: Home Assistant REST API.

## 7. Preparar a API própria

Crie uma pasta no computador Linux, salve o código das páginas seguintes como `api_tomada.py` e instale as dependências. O código consulta o Home Assistant a cada 60 segundos e grava amostras em SQLite. Ele não envia comandos à tomada.

```
python3 -m venv .venv
. .venv/bin/activate
pip install fastapi uvicorn
```

# Código da API • parte 1 de 2

Copie as duas partes em sequência para o arquivo `api_tomada.py`.

```
import hmac
import json
import os
import sqlite3
import threading
from contextlib import asynccontextmanager, closing
from datetime import datetime, timezone
from urllib.error import HTTPError, URLError
from urllib.request import Request, urlopen

from fastapi import FastAPI, Header, HTTPException


HA_URL = os.environ["HA_URL"].rstrip("/")
HA_TOKEN = os.environ["HA_TOKEN"]
API_KEY = os.environ["API_KEY"]
DB_PATH = os.getenv("DB_PATH", "tomadas.sqlite3")
INTERVALO = max(30, int(os.getenv("INTERVALO_SEG", "60")))
ENTIDADES = {
    "ligada": os.environ["HA_SWITCH_ENTITY"],
    "energia": os.getenv("HA_ENERGY_ENTITY", ""),
    "corrente": os.getenv("HA_CURRENT_ENTITY", ""),
    "potencia": os.getenv("HA_POWER_ENTITY", ""),
    "tensao": os.getenv("HA_VOLTAGE_ENTITY", ""),
}
parar = threading.Event()


def consultar(identificador):
    req = Request(
        f"{HA_URL}/api/states/{identificador}",
        headers={"Authorization": f"Bearer {HA_TOKEN}"},
    )
    with urlopen(req, timeout=8) as resposta:
        dado = json.load(resposta)
    return {
        "valor": dado["state"],
        "unidade": dado.get("attributes", {}).get("unit_of_measurement"),
        "atualizado_em": dado.get("last_updated"),
    }


def banco():
    con = sqlite3.connect(DB_PATH)
    con.execute("PRAGMA journal_mode=WAL")
    con.execute("CREATE TABLE IF NOT EXISTS amostras "
                "(id INTEGER PRIMARY KEY, instante TEXT NOT NULL, dados TEXT NOT NULL)")
    con.commit()
    return con
```

## Código da API • parte 2 de 2

```
def coletar():
    while not parar.is_set():
        dados = {}
        for campo, entidade in ENTIDADES.items():
            if not entidade:
                continue
            try:
                dados[campo] = consultar(entidade)
            except (HTTPError, URLError, TimeoutError, ValueError, KeyError) as erro:
                dados[campo] = {"erro": type(erro).__name__}
        instante = datetime.now(timezone.utc).isoformat()
        with closing(banco()) as con:
            con.execute("INSERT INTO amostras(instante, dados) VALUES (?, ?)",
                        (instante, json.dumps(dados, ensure_ascii=False)))
            con.commit()
        parar.wait(INTERVALO)


@asynccontextmanager
async def ciclo(app):
    with closing(banco()):
        pass
    tarefa = threading.Thread(target=coletar, daemon=True)
    tarefa.start()
    yield
    parar.set()
    tarefa.join(timeout=10)


app = FastAPI(title="API local de tomadas", lifespan=ciclo)


def autorizar(chave):
    if not chave or not hmac.compare_digest(chave, API_KEY):
        raise HTTPException(status_code=401, detail="Chave invalida")


@app.get("/v1/tomadas/ambiente-01/atual")
def atual(x_api_key: str | None = Header(default=None)):
    autorizar(x_api_key)
    with closing(banco()) as con:
        linha = con.execute("SELECT instante, dados FROM amostras "
                            "ORDER BY id DESC LIMIT 1").fetchone()
    if not linha:
        raise HTTPException(status_code=503, detail="Sem amostras")
    return {"ambiente": "ambiente-01", "instante": linha[0],
            "leituras": json.loads(linha[1])}


@app.get("/v1/tomadas/ambiente-01/historico")
def historico(limite: int = 100, x_api_key: str | None = Header(default=None)):
    autorizar(x_api_key)
    limite = min(max(limite, 1), 1000)
    with closing(banco()) as con:
        linhas = con.execute("SELECT instante, dados FROM amostras "
                             "ORDER BY id DESC LIMIT ?", (limite,)).fetchall()
    return [{"instante": t, "leituras": json.loads(d)} for t, d in linhas]
```

As unidades são as que o Home Assistant informar. Não converta automaticamente Wh, kWh, W, V ou mA antes de validar a entidade.

## 8. Configurar e iniciar

No mesmo terminal, informe os IDs reais vistos no passo 5. Se algum sensor não existir, deixe sua variável ausente; o campo será omitido.

```
export HA_URL="http://<IP_HA>:8123"
export HA_TOKEN="<TOKEN_DO_HOME_ASSISTANT>"
export API_KEY="<CHAVE_LONGA_ALEATORIA>"
export HA_SWITCH_ENTITY="switch.<ID_REAL>"
export HA_ENERGY_ENTITY="sensor.<ID_REAL>"
export HA_CURRENT_ENTITY="sensor.<ID_REAL>"
export HA_POWER_ENTITY="sensor.<ID_REAL>"
export HA_VOLTAGE_ENTITY="sensor.<ID_REAL>"
uvicorn api_tomada:app --host 127.0.0.1 --port 8000
```

Para gerar uma chave: `python3 -c "import secrets; print(secrets.token_urlsafe(32))"`. Guarde token e chave fora do código e não publique a porta 8000 na internet.

## 9. Testar a API

```
curl -H "X-API-Key: <SUA_CHAVE>" \
+  http://127.0.0.1:8000/v1/tomadas/ambiente-01/atual

curl -H "X-API-Key: <SUA_CHAVE>" \
+  "http://127.0.0.1:8000/v1/tomadas/ambiente-01/historico?limite=10"
```

- Ligue uma carga conhecida e confirme que potência/corrente mudam. Desligue-a e confirme o retorno próximo de zero.
- Deixe o coletor ativo por pelo menos uma hora; confira se há amostras regulares e se o campo de energia, quando disponível, evolui coerentemente.
- Teste sem internet por alguns minutos, mantendo a LAN ativa. Confirme se as leituras locais continuam; alguns firmwares podem depender da nuvem para permanecer responsivos.
- Confirme que uma chave de API incorreta recebe HTTP 401. Faça cópia do arquivo SQLite e documente a recuperação após reinicialização.

## Como interpretar o retorno

A API devolve as leituras como texto e conserva a unidade mostrada pelo Home Assistant. Assim evitamos converter valores de um firmware sem conhecer a escala real. Um exemplo ilustrativo de resposta atual é:

```
{
  "ambiente": "ambiente-01",
  "instante": "2026-09-16T15:00:00+00:00",
  "leituras": {
    "ligada": {"valor": "on", "unidade": null},
    "potencia": {"valor": "83.4", "unidade": "W"},
    "tensao": {"valor": "220.1", "unidade": "V"},
    "corrente": {"valor": "379", "unidade": "mA"}
  }
}
```

Esse JSON é apenas um exemplo de formato; os IDs, valores e campos disponíveis dependerão da tomada piloto.

### Consumo acumulado e qualidade dos dados

- Se a entidade de energia estiver presente, determine se ela representa incremento, total acumulado ou outro intervalo. O DP 17 da solução Tuya é descrito como energia adicionada. Compare a leitura com uma carga estável ao longo de uma hora.
- Se houver apenas potência, uma estimativa pode ser calculada: kWh ≈ soma de (W × horas entre amostras) / 1000. Exemplo: 100 W constantes durante 2 h ≈ 0,2 kWh. Registre períodos sem leitura como lacunas.
- A amostra tem dois horários: "instante" é quando nosso coletor consultou; "atualizado_em" é quando o Home Assistant viu a última mudança. Se o segundo for antigo, a leitura pode estar desatualizada.

### Diagnóstico rápido

| Sintoma | Verificação |
|---|---|
| Tomada não aparece | Conta do app, IP e alcance entre sub-redes; depois, chave e versão local. |
| Sensor sem valor | Entidade desativada/oculta; DP ausente na unidade ou firmware. |
| HTTP 401 | Confira a chave X-API-Key; não a confunda com o token HA. |
| Erro em uma leitura | Confira HA_URL, token, ID da entidade e /api/states/ID. |

## Decisão após o piloto

| Resultado | Próxima ação |
|---|---|
| DP 17 e DP 19 disponíveis | Ampliar para mais tomadas; validar semântica do DP 17 e retenção. |
| Só potência disponível | Calcular energia aproximada por integração temporal, com lacunas sinalizadas. |
| Só estado disponível | A tomada piloto não atende à meta de consumo local; avaliar outro firmware/modelo. |
| Conexão local instável | Revisar chave, versão do protocolo, Wi-Fi, VLAN e número de conexões. |

## Cuidados para expansão

- Coloque o coletor em uma rede com acesso apenas às tomadas necessárias. Dê nomes que associem cada ID à sala e ao circuito.
- Mantenha segredos fora do repositório. Recadastrar uma tomada pode trocar a chave local.
- Planeje backup, monitoramento de indisponibilidade, sincronização de horário e política de retenção do SQLite antes de incluir toda a empresa.
- A API piloto é somente de leitura. Controle remoto de tomadas deve ser projetado e autorizado separadamente.

## Fontes consultadas

NovaDigital, manual T20A e página T20MC.

tuya-local, perfil T20A e instruções de instalação.

Tuya, definição dos pontos de medição. Home Assistant, REST API. HACS, instalação.

Limite deste documento: é um procedimento de implantação, não uma medição feita em sua empresa. O conjunto definitivo de campos e a estabilidade só ficam confirmados após o passo 9.